Orientation Tracking for a Rigid Body using IMU Data

Dhruv Shah
PhD Student, Mechanical and Aerospace Engineering
University of California, San Diego
dhshah@ucsd.edu

ECE 276A: Sensing & Estimation in Robotics
February 27, 2026

1 Introduction

Simultaneous Localization and Mapping (SLAM) is a fundamental capability that enables an autonomous robot to navigate and map an unknown environment concurrently. This framework is highly prominent in a variety of modern applications, including surveillance systems, autonomous driving, and complex robotic tasks. In this project, we implement a full LiDAR-based SLAM pipeline using a combination of encoder and IMU odometry, 2D LiDAR scans, and RGBD measurements for a differential-drive robot.

We tackle the SLAM process in a sequential manner. First, we establish a baseline for the robot's trajectory by calculating odometry using the IMU and wheel encoder sensors. While this high-frequency dead-reckoning provides a reasonable short-term pose estimate, it inherently suffers from unbounded drift over time due to sensor noise and wheel slippage.

To refine this local estimate, we incorporate high-resolution 2D LiDAR point clouds. By applying the Iterative Closest Point (ICP) algorithm—initialized with the encoder and IMU odometry—we compute precise relative pose transformations between consecutive scans. By design, this significantly tightens the local odometry and corrects minor continuous drift.

Finally, to address long-term error accumulation, we leverage pose graph optimization and loop closure detection. By actively detecting loop closures—identifying instances where the robot revisits a previously mapped location and crosses its own path—we introduce strong global constraints into our optimization problem. This allows us to optimize the entire trajectory, successfully eliminating long-term drift and generating a globally consistent 2D occupancy grid map of the environment.

2 Problem Formulation

Our primary objective is to accurately estimate the robot's trajectory $X = \{x_0, x_1, \ldots, x_T\}$ and the surrounding environment map. The state $x_t = (p_t, \theta_t) \in SE(2)$ represents the 2D position and orientation of the robot at time step t. We represent the environment as a 2D occupancy grid map M, where each cell m_i holds the probability of being occupied based on LiDAR measurements. We formulate this mathematically across four core components:

2.1 Encoder and IMU Odometry

We use the differential-drive robot kinematics to predict the next pose based on control inputs $u_t = (v_t, \omega_t)$, which are derived from the wheel encoders (linear velocity) and the IMU (angular

velocity):

$$\begin{aligned} \dot{p}_x = v_t \cos(\theta_t), \quad \dot{p}_y = v_t \sin(\theta_t), \quad \dot{\theta} = \omega_t \\ v_t = \frac{r}{2}(\omega_{left} + \omega_{right}), \quad \omega_t = \frac{r}{L}(\omega_{right} - \omega_{left}) \end{aligned} \tag{1}$$

where r is the wheel radius and L is the distance between the wheels. w_{left} and w_{right} are the angular velocities of the left and right wheels, measured by the wheel encoders. This provides a first-order estimate of the robot's trajectory, but it is subject to drift over time. The problem is to compute the sequence of poses X given the control inputs $U = \{u_0, u_1, \ldots, u_{T-1}\}$.

2.2 Point Cloud Registration

Let P_t and P_{t+1} be consecutive 2D LiDAR point clouds. The problem is to find the the point assignment $c(i)$ and the relative rigid-body transformation $T_{t,t+1} \in SE(2)$ that optimally aligns the two scans. This can be formulated as an optimization problem:

$$T^*_{t,t+1},\, c^*(\cdot) \equiv \arg\min \sum_{i=1}^{N} \|p_{t,i} - T \cdot p_{t+1,\,c(i)}\|^2 \tag{2}$$

where $p_{t,i} \in P_t$ and $c(i)$ denotes the corresponding point index in the target cloud P_{t+1}. This is a combinatorial optimization problem. The relative transformation $T_{t,t+1}$ can then be used to update the global trajectory estimate:

$$x_{t+1} = T_{t,t+1} \oplus x_t \tag{3}$$

where $\oplus$ denotes the composition of poses in $SE(2)$. This yields a refined trajectory that corrects for local drift, but it still may accumulate error over long distances.

2.3 Occupancy Mapping

Using the sequence of optimized poses $X = \{x_0, x_1, \ldots, x_T\}$ and LiDAR range measurements $z_{1:T}$, we aim to build a 2D occupancy grid. This is done for each grid cell m_i, by recursively updating the posterior probability of occupancy using a log-odds formulation:

$$L(m_i|z_{1:t}, x_{1:t}) = L(m_i|z_{1:t-1}, x_{1:t-1}) + \text{inv_sensor_model}(m_i, z_t, x_t) - L_0 \tag{4}$$

where $L(m_i)$ is the log-odds ratio and inv_sensor_model returns the probability of the cell being occupied based on the ray-casting geometry from the robot's pose x_t. L_0 is the prior log-odds of occupancy. The problem is to compute the occupancy grid map M that best represents the environment given the optimized trajectory and sensor measurements.

2.4 Pose Graph Optimization

To correct long-term drift, we formulate the SLAM problem as a factor graph. The nodes in the graph represent the robot poses x_t, and the edges denote the relative pose constraints derived from ICP odometry and loop closures. We find the globally optimal trajectory X^* by minimizing the Mahalanobis distance of the error residuals:

$$X^* = \arg\min_X \sum_{(i,j)\in\mathcal{E}} \|z_{i,j} \ominus T_{i,j}(X)\|^2_{\Sigma_{ij}} \tag{5}$$

where $\mathcal{E}$ represents the set of all sequential and loop-closure edges, z_{ij} is the relative measurement constraint between pose i and j, $T_{i,j}(X)$ is the predicted relative transformation based on the current trajectory estimate, $\ominus$ computes the manifold error in $SE(2)$, and Σ_{ij} is the associated measurement covariance matrix.

3 Technical Approach

Our SLAM pipeline processes the sensor modalities in a sequential manner, gradually refining the trajectory estimation from a local to a global scale. The technical implementation is broken down into the following stages.

3.1 Odometry Initialization

First, we process the encoder and IMU data to establish a baseline trajectory. The linear velocity is calculated from the left and right wheel encoder ticks, while the angular velocity is read directly from the IMU's vertical axis (z-axis). The angular velocity is also cross-validated with the encoder-based estimate to ensure consistency. This is done using the robot kinematics (1). The parameters r and L are determined from the robot's specifications.

Using (1), we integrate these control inputs using a first-order Euler approximation. This yields an initial dead-reckoning pose estimate at each timestamp, providing a smooth but drifting prior for our scan-matching algorithm. The linear and angular velocity profiles are plotted to verify the consistency between the encoder and IMU measurements, ensuring that the IMU is providing reliable orientation information. This is summarized in Figure 1. The resulting trajectory and occupancy grid map generated from this raw odometry are shown in Figure 2. The details for generating the occupancy grid map from the robot odometry are described in the Section 3.3.

3.2 Point Cloud Registration via ICP

To mitigate the accumulation of drift from wheel slippage and noisy IMU integration, we utilize the Iterative Closest Point (ICP) algorithm to perform scan matching between consecutive LiDAR sweeps. As described in equation (2), the ICP algorithm solves this problem by iteratively finding the closest point correspondences between two scans and optimizing for the rigid-body transformation that minimizes the mean squared error between these correspondences. This makes the algorithm very dependent on the initial guess, which is why we use the odometry-based relative pose estimate to initialize the ICP transformation matrix.

We warm up the ICP implementation by testing it on synthetic point clouds generated from the canonical 3D models of the drill and liquid container objects. In order to reach a good local minimum, we utilize several random initializations of the transformation matrix. We also drop points from the liquid container point cloud to speed up convergence and avoid local minima, since the liquid container has a lot of points and can cause slow convergence in ICP. The results of the ICP registration for both objects are shown in Figure 3.

Next, we apply the ICP algorithm to the real LiDAR scans collected by the robot. First, the 2D LiDAR range and bearing measurements are converted into local Cartesian coordinates. We utilize the LIDAR's position and orientation relative to the robot's optical center to transform these points into the robot's local frame.

To LIDAR Cartesian: Converting 2D LiDAR range and bearing measurements into world Cartesian coordinates involves a sequence of standard 2D rigid body transformations. For each scan point, we receive a range measurement (r) and a bearing/angle (α). We convert this polar coordinate into a local Cartesian coordinate (x_L, y_L) in the LiDAR's own reference frame using basic trigonometry:

$$x_L = r\cos(\alpha), \quad y_L = r\sin(\alpha) \tag{6}$$

To Body Cartesian: The LiDAR is physically mounted on the robot at a specific offset. Let the LiDAR's position relative to the robot's center be $(x_{\text{offset}}, y_{\text{offset}})$ and its rotational offset be θ_{offset}. To transform the point from the LiDAR's frame to the robot's local reference frame (x_R, y_R), we apply a 2D rotation and translation:

$$x_R = x_{\text{offset}} + x_L \cos(\theta_{\text{offset}}) - y_L \sin(\theta_{\text{offset}}) \tag{7}$$
$$y_R = y_{\text{offset}} + x_L \sin(\theta_{\text{offset}}) + y_L \cos(\theta_{\text{offset}}) \tag{8}$$

Note, for our robot, the LiDAR is aligned with the robot heading, so $\theta_{\text{offset}} = 0$. This transformation ensures that all points are represented in a consistent local frame centered on the robot, which is essential for accurate scan matching and mapping.

To World Cartesian: Finally, to map the point to the global "world" frame, we must account for the robot's current pose in the world. Let the robot's global coordinates be (x_w, y_w) and its global heading (yaw) be θ_w. We apply another rotation and translation to map the point (x_R, y_R) to the world coordinates $(x_{\text{world}}, y_{\text{world}})$:

$$x_{\text{world}} = x_w + x_R \cos(\theta_w) - y_R \sin(\theta_w) \tag{9}$$
$$y_{\text{world}} = y_w + x_R \sin(\theta_w) + y_R \cos(\theta_w) \tag{10}$$

The complete transformation from the LiDAR polar coordinates to the world frame is:

$$\begin{bmatrix} x_{\text{world}} \\ y_{\text{world}} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta_w) & -\sin(\theta_w) & x_w \\ \sin(\theta_w) & \cos(\theta_w) & y_w \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta_{\text{offset}}) & -\sin(\theta_{\text{offset}}) & x_{\text{offset}} \\ \sin(\theta_{\text{offset}}) & \cos(\theta_{\text{offset}}) & y_{\text{offset}} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r\cos(\alpha) \\ r\sin(\alpha) \\ 1 \end{bmatrix} \tag{11}$$

For each pair of consecutive scans, the encoder odometry-based relative pose estimate is used to initialize the ICP transformation matrix. We iteratively compute the nearest-neighbor point correspondences and solve for the optimal $SE(2)$ rigid-body transformation using the Kabsch Algorithm.

3.3 Occupancy Grid Mapping

We also aim to construct a 2D occupancy grid to map the environment. We define a discrete Cartesian grid where each cell stores a log-odds value, representing the probability of the cell being an obstacle. Using the optimized poses, we transform each LiDAR point from the robot's local frame into the global map frame. We employ a ray-casting technique (Bresenham's line algorithm) to trace the path from the robot's optical center to the measured laser hit point. The log-odds probabilities for the grid cells along the ray are decreased (representing free space), while the log-odds for the end-point cell are increased (representing an occupied obstacle). We apply saturation limits to the log-odds values to prevent overconfidence in the sensor readings. This process is repeated for all LiDAR points across all time steps, resulting in a comprehensive occupancy grid map that reflects the spatial layout of the environment. The occupancy grid map generated from the ICP-optimized trajectory is shown in Figure 4.

3.4 Pose Graph Optimization and Loop Closure

To achieve global consistency and correct long-term drift, we formulate the SLAM problem as a factor graph and solve it using the GTSAM optimization library. The system states (2D poses) are instantiated as variables in the graph. The relative transformations computed from our ICP scan-matching step serve as sequential odometry factors (edges) connecting consecutive poses.

To introduce loop closures, we implement a fixed interval loop closure and a proximity detection module. We introduce a loop closure every 10 seconds, which adds a constraint between the current pose and the pose from 10 seconds ago. This helps to anchor the trajectory and reduce drift. Furthermore, as the robot navigates, we evaluate the spatial distance between the current pose and historical poses. If this is below a certain threshold, we attempt to register the current LiDAR scan with the historical scan using ICP. For both loop closure candidates, if the registration converges with a sufficiently low fitness score (mean squared error), we validate the loop closure and insert a new global constraint edge between the corresponding non-sequential nodes in the factor graph. Finally, we optimize the entire graph using the Levenberg-Marquardt optimizer in GTSAM. The resulting globally optimal poses are then used to regenerate the final, drift-free occupancy grid map. The trajectory and occupancy grid map generated from the GTSAM-optimized trajectory are shown in Figure 5.

4 Results

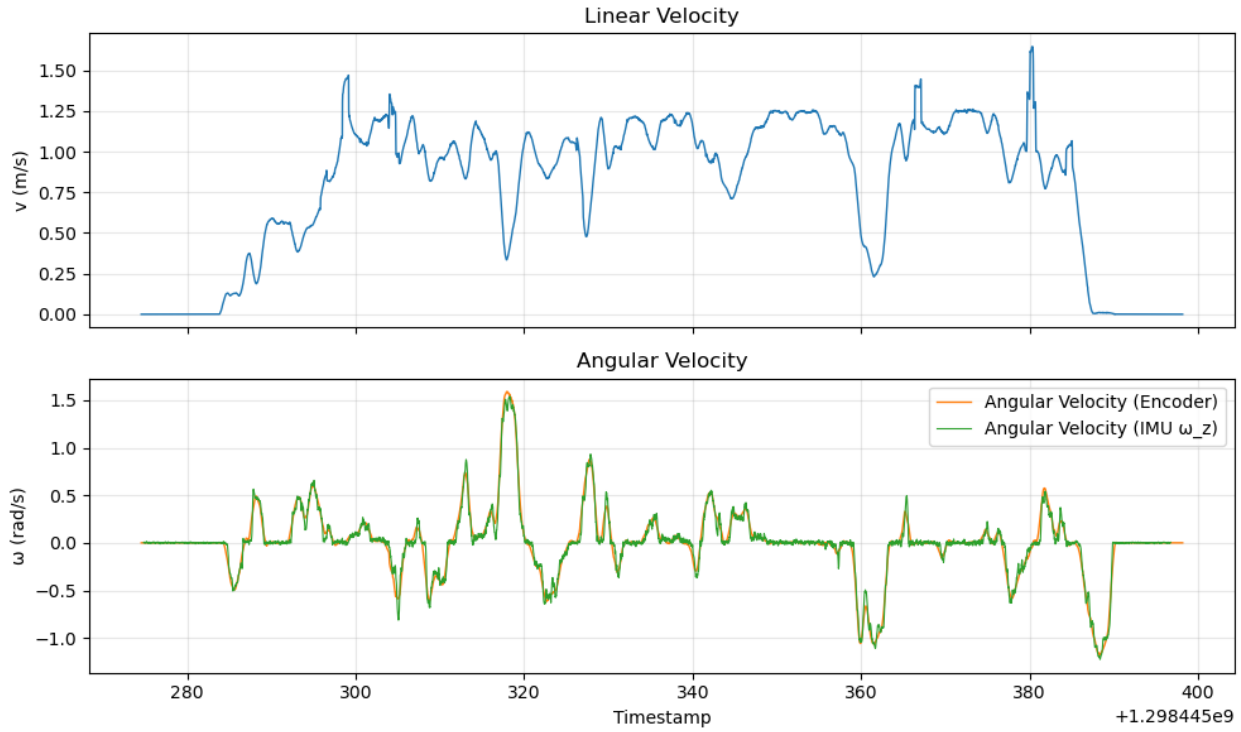
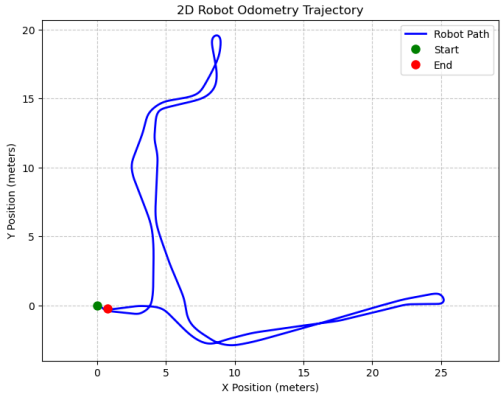
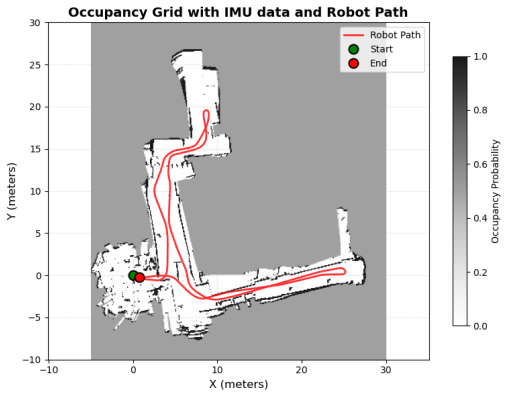


Figure 1: Linear velocity (top) obtained from the encoder data and angular velocity (bottom) obtained from the encoder and IMU. The angular velocity matches closely between the encoder and IMU, indicating that the IMU is providing reliable orientation information.

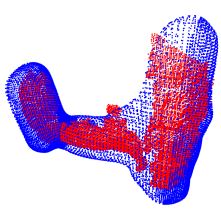


(a) Robot trajectory

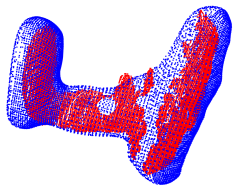


(b) Occupancy grid map

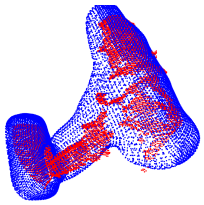
Figure 2: The robot trajectory (a) and occupancy grid map (b) generated using the IMU and encoder odometry. The trajectory shows significant drift over time, especially in the yaw direction, which leads to a distorted occupancy grid map with misaligned features. Specifically near the start and end points. This highlights the need for further optimization using scan matching and loop closure to correct for this drift.



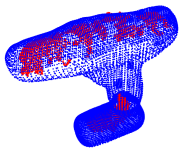
(a) Drill 1



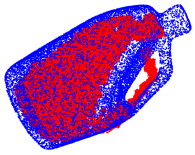
(b) Drill 2



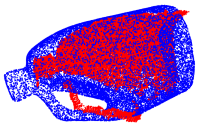
(c) Drill 3



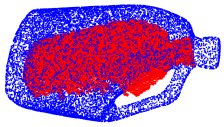
(d) Drill 4



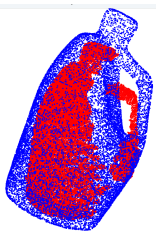
(e) Container 1



(f) Container 2

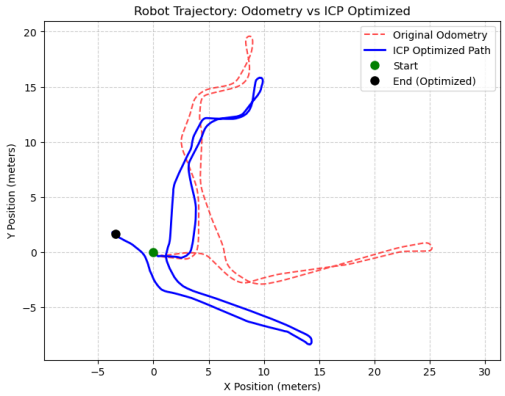


(g) Container 3

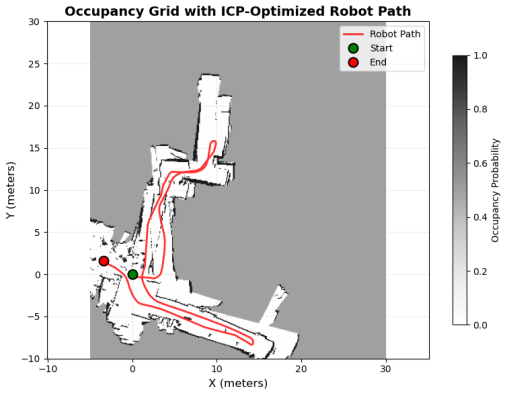


(h) Container 4

Figure 3: The results of ICP registration for the drill (top row) and liquid container (bottom row) objects. Each column shows the aligned point cloud (red) and target point cloud (blue). The ICP algorithm successfully aligns the point clouds, demonstrating its effectiveness in refining pose estimates based on LiDAR data. However, the alignment is not perfect, especially for the liquid container, which has a more complex geometry. We utilize several random intializations to ensure that we find a good local minimum for the ICP optimization. We also drop points from the liquid container point cloud to speed up convergence and avoid local minima.

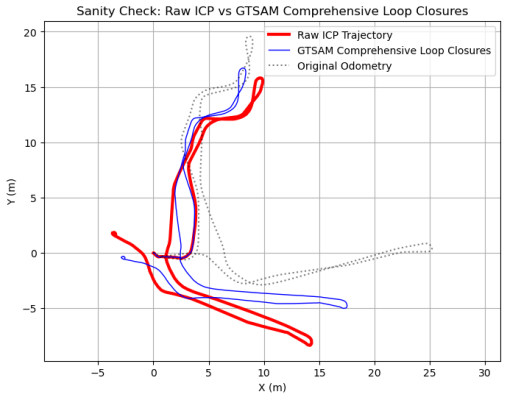


(a) Robot trajectory

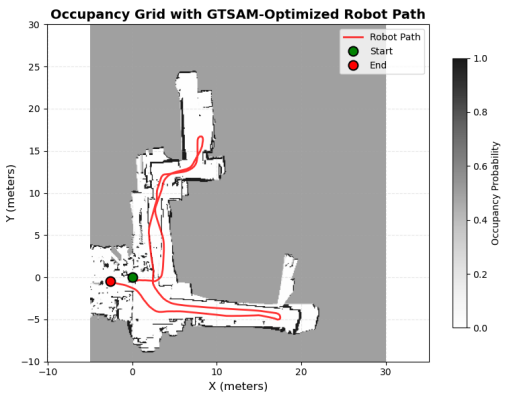


(b) Occupancy grid map

Figure 4: The robot trajectory (a) and occupancy grid map (b) generated using ICP-optimized odometry. The drift over time has reduced significantly. This is clear from the accurate mapping of the small room that the robot begins in and ends in. However, there is still some distortion in the occupancy grid map, particularly in the larger open areas, which may be due to residual errors in the orientation estimates and the limited number of loop closures detected. This highlights the importance of incorporating global optimization and loop closure constraints to further refine the trajectory and map.



(a) Robot trajectory



(b) Occupancy grid map

Figure 5: The robot trajectory (a) and occupancy grid map (b) generated using GTSAM-optimized odometry. The trajectory is now much more accurate, with significantly reduced drift. The occupancy grid map seems to be globally consistent, with well-aligned features and clear mapping of the environment. The loop closures have successfully corrected for long-term drift, resulting in a more accurate representation of the space.